

Data and Artificial Intelligence

Cyber Shujaa Program

Week 8 Assignment Classification Models (Wine Dataset)

Student Name: STEPHANNIE AWUOR LUMUMBA

Student ID: CS-DA02-25097

Table of Contents

Data and Artificial Intelligence	1
Cyber Shujaa Program.....	1
Week 8 Assignment Classification Models (Wine Dataset).....	1
1. Introduction.....	2
2. Task Completion	2
2.1 Data Loading and Exploration	2
2.2 Data Preparation	6
2.3 Model Training	7
.....	7
2.4 Model Evaluation	7
2.6 Comparison of Results	8
3. Link to code:	9
https://www.kaggle.com/code/stephanieawuor/stephannie-lumumba-25097-classification-model...	9
4. Conclusion	9

1. Introduction

This assignment focuses on implementing and comparing multiple supervised machine learning **classification models** using the **Wine dataset** from scikit-learn.

The main objective was to explore the dataset, prepare the data, train six different classification models, and evaluate their performance using standard metrics such as **accuracy, precision, recall, F1-score**, and **confusion matrices**.

The models implemented were:

1. Logistic Regression
2. Decision Tree
3. Random Forest
4. k-Nearest Neighbors (KNN)
5. Naive Bayes
6. Support Vector Machine (SVM)

Through this assignment, I aimed to gain a deeper understanding of how different algorithms perform on the same dataset under similar conditions and to learn how to interpret the results effectively.

2. Task Completion

2.1 Data Loading and Exploration

The Wine dataset was loaded using `load_wine()` from scikit-learn. The dataset consists of 178 samples with 13 numerical features representing chemical properties of different wine cultivars.

I conducted **Exploratory Data Analysis (EDA)** using Pandas and Seaborn to understand the structure and distribution of data.

- Checked for missing values (none were found)
- Generated summary statistics
- Plotted class distributions and feature correlations



+

 Create

Home

Competitions

Datasets

Models

Benchmarks

Game Arena

<> Code

Discussions

Stephannie.Lumumba_25097_Classification_Model

Notebook

Input

Output

Logs

Comments (0)

Settings

In [1]:

```
# Importing libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Version 1 of 1

Runtime

27s

Tags

Add Tags

Kaggle uses cookies from Google to deliver and enhance the quality of its services and to analyze traffic.

[Learn more](#)
[OK, Got it](#)



+

 Create

Home

Competitions

Datasets

Models

Benchmarks

Game Arena

<> Code

Discussions

Notebook

Input

Output

Logs

Comments (0)

Settings

In [2]:

```
# Scikit-learn imports
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix
)
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
```

Language

Python

0

Share

Edit



+

 Create

Home

Competitions

Datasets

Models

Benchmarks

Game Arena

<> Code

Discussions

Notebook

Input

Output

Logs

Comments (0)

Settings

In [3]:

```
# Set plotting style
sns.set(style='whitegrid')
plt.rcParams['figure.figsize'] = (8,5)
```

In [4]:

```
# Load Wine dataset
wine = load_wine()
X = pd.DataFrame(wine.data, columns=wine.feature_names)
y = pd.Series(wine.target, name='target')

# Combine features + target for exploration
df = pd.concat([X, y], axis=1)
df.head()
```

0

Share

Edit

+

 Create

Home

Competitions

Datasets

Models

Benchmarks

Game Arena

Code

Discussions

0

Share

Edit

Notebook

Input

Output

Logs

Comments (0)

Settings

Out[4]:

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	flavanoids	nonflavanol
0	14.23	1.71	2.43	15.6	127.0	2.80	3.06	0.28
1	13.20	1.78	2.14	11.2	100.0	2.65	2.76	0.26
2	13.16	2.36	2.67	18.6	101.0	2.80	3.24	0.30
3	14.37	1.95	2.50	16.8	113.0	3.85	3.49	0.24
4	13.24	2.59	2.87	21.0	118.0	2.80	2.69	0.39

In [5]:

```
# Basic info
df.info()
```

+

 Create

Home

Competitions

Datasets

Models

Benchmarks

Game Arena

Code

Discussions

View Active Events

0

Share

Edit

Notebook

Input

Output

Logs

Comments (0)

Settings

0	alcohol	178 non-null	float64	
1	malic_acid	178 non-null	float64	
2	ash	178 non-null	float64	
3	alcalinity_of_ash	178 non-null	float64	
4	magnesium	178 non-null	float64	
5	total_phenols	178 non-null	float64	
6	flavanoids	178 non-null	float64	
7	nonflavanoid_phenols	178 non-null	float64	
8	proanthocyanins	178 non-null	float64	
9	color_intensity	178 non-null	float64	
10	hue	178 non-null	float64	
11	od280/od315_of_diluted_wines	178 non-null	float64	
12	proline	178 non-null	float64	
13	target	178 non-null	int64	

dtypes: float64(13), int64(1)

memory usage: 19.6 KB

+

 Create

Home

Competitions

Datasets

Models

Benchmarks

Game Arena

Code

Discussions

View Active Events

0

Share

Edit

Notebook

Input

Output

Logs

Comments (0)

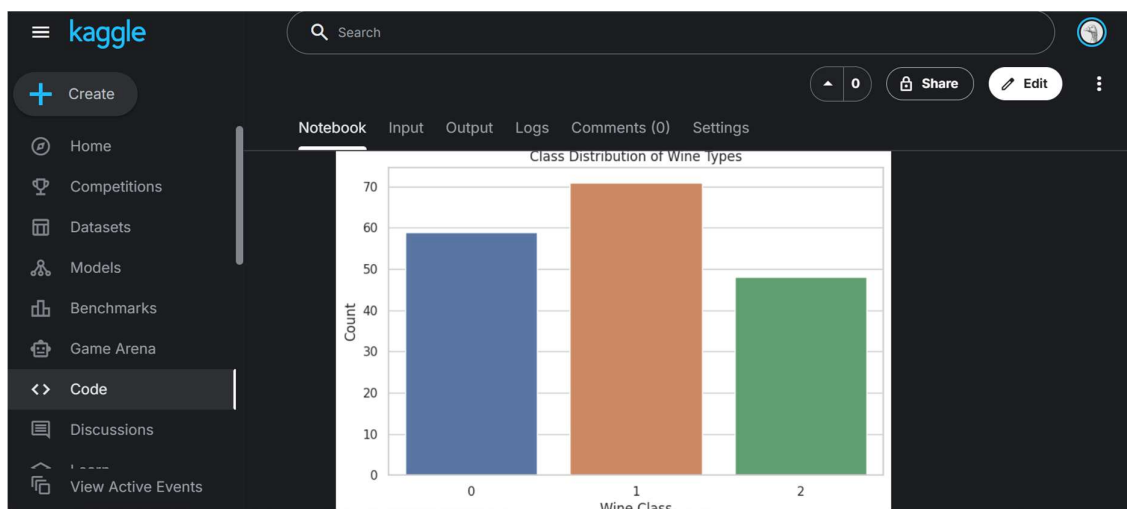
Settings

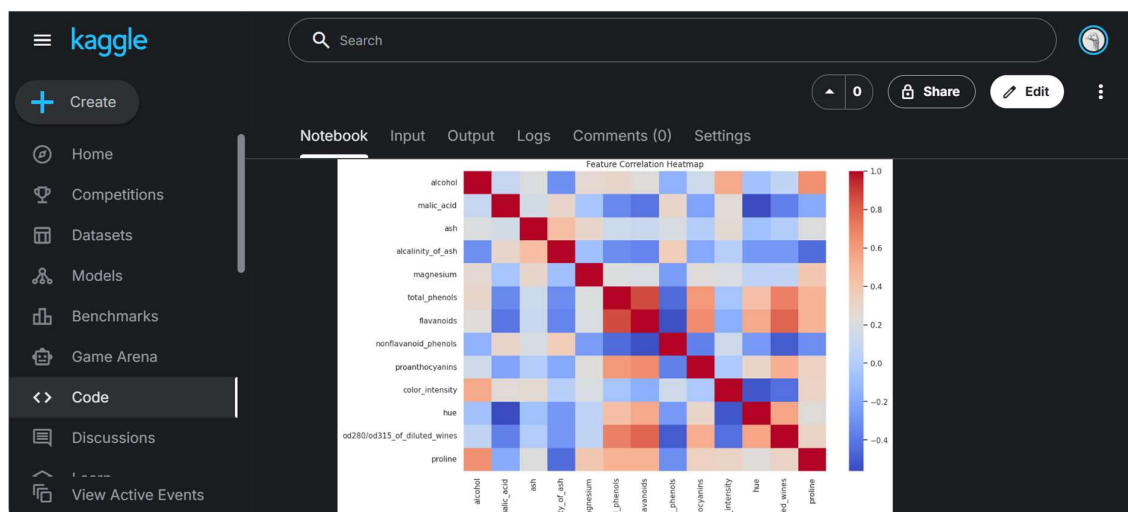
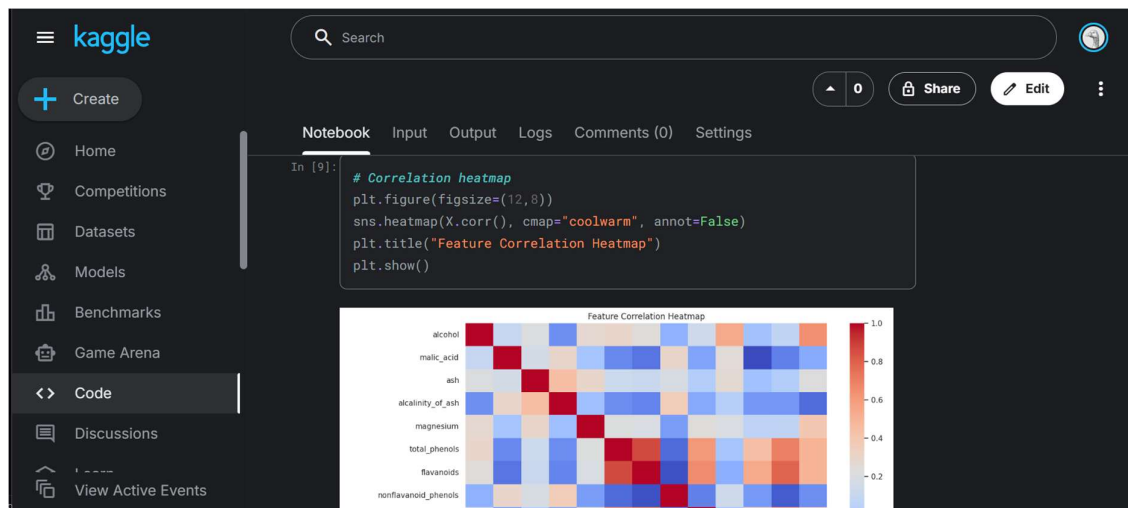
In [6]:

```
# Summary statistics
df.describe()
```

Out[6]:

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	flavanoids
count	178.000000	178.000000	178.000000	178.000000	178.000000	178.000000	178.000000
mean	13.000618	2.336348	2.366517	19.494944	99.741573	2.295112	2.0292
std	0.811827	1.117146	0.274344	3.339564	14.282484	0.625851	0.9988
min	11.030000	0.740000	1.360000	10.600000	70.000000	0.980000	0.3400
25%	12.362500	1.602500	2.210000	17.200000	88.000000	1.742500	1.2050
50%	13.050000	1.865000	2.360000	19.500000	98.000000	2.355000	2.1350
75%	13.677500	3.082500	2.557500	21.500000	107.000000	2.800000	2.8750
max	14.830000	5.800000	3.230000	30.000000	162.000000	3.880000	5.0800

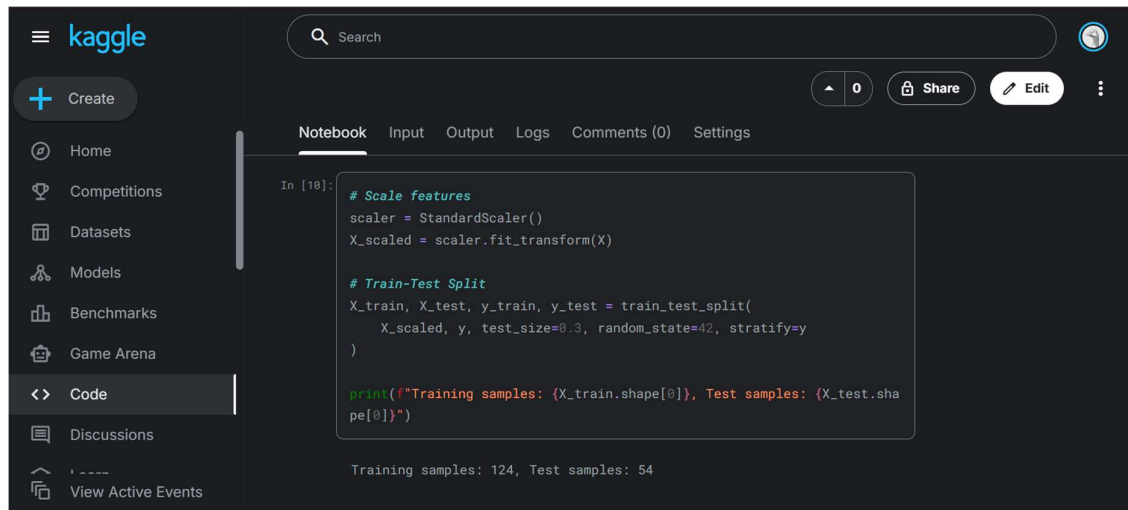




2.2 Data Preparation

Before training the models, all numerical features were standardized using **StandardScaler** to ensure uniform scale.

The dataset was split into **70% training** and **30% testing** sets using `train_test_split()` with `random_state=42` to ensure reproducibility.



```

In [10]: # Scale features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.3, random_state=42, stratify=y
)

print(f"Training samples: {X_train.shape[0]}, Test samples: {X_test.shape[0]}")

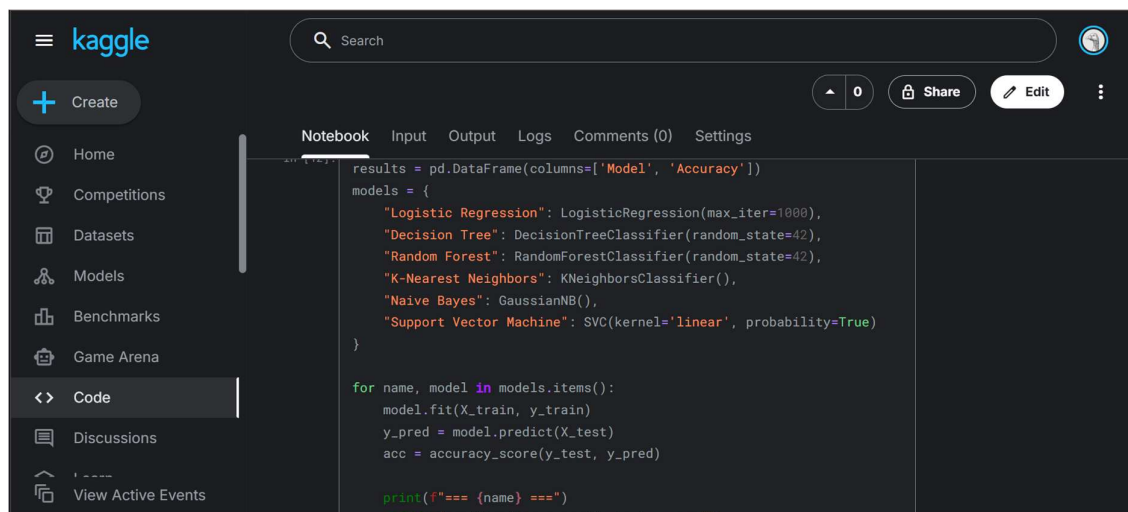
```

Training samples: 124, Test samples: 54

2.3 Model Training

Six classification algorithms were implemented and trained using the same training dataset. Each model was fitted and then used to predict outcomes on the test data. The algorithms used included:

- **Logistic Regression (LR)**
- **Decision Tree (DT)**
- **Random Forest (RF)**
- **K-Nearest Neighbors (KNN)**
- **Naive Bayes (NB)**
- **Support Vector Machine (SVM)**



```

results = pd.DataFrame(columns=['Model', 'Accuracy'])
models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Random Forest": RandomForestClassifier(random_state=42),
    "K-Nearest Neighbors": KNeighborsClassifier(),
    "Naive Bayes": GaussianNB(),
    "Support Vector Machine": SVC(kernel='linear', probability=True)
}

for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)

    print(f"=== {name} ===")
    print(f"Classification report for {name}:")

```

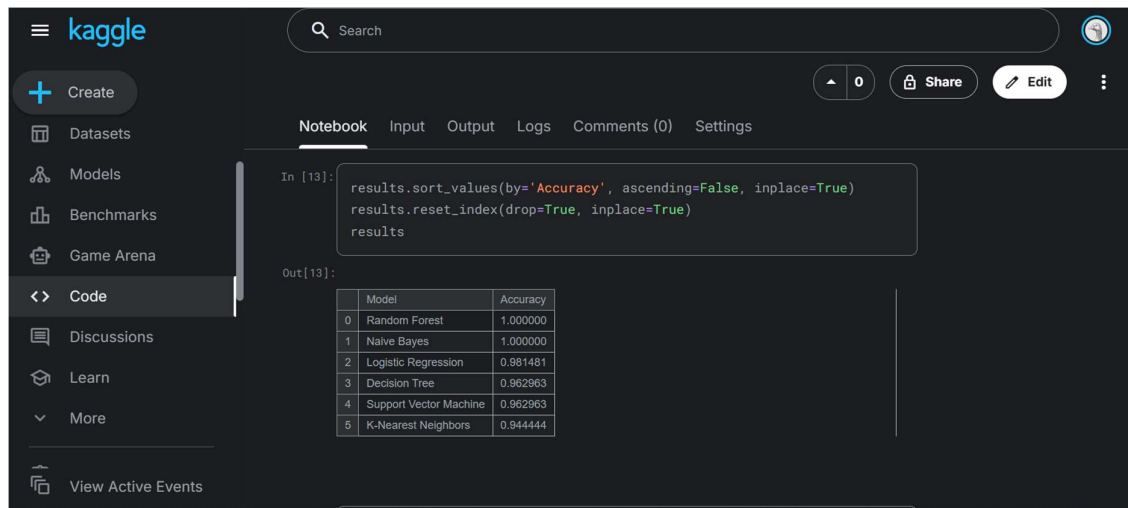
2.4 Model Evaluation

For each model, the following metrics were calculated:

- **Accuracy Score**
- **Precision, Recall, and F1-Score**
- **Confusion Matrix**

Heatmaps of confusion matrices were plotted to visualize performance across the three wine classes.

The evaluation results were compiled into a results table for comparison.

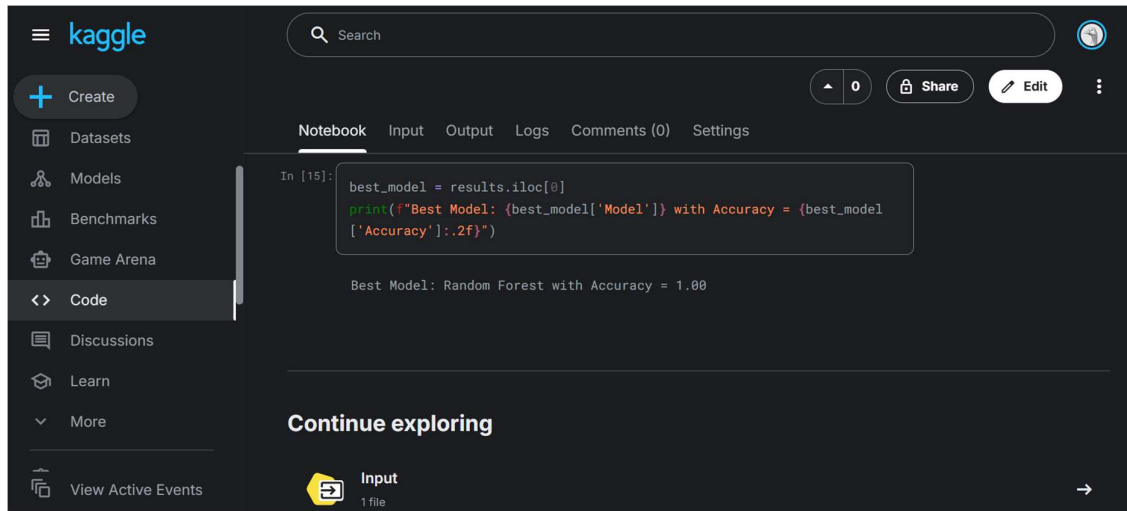
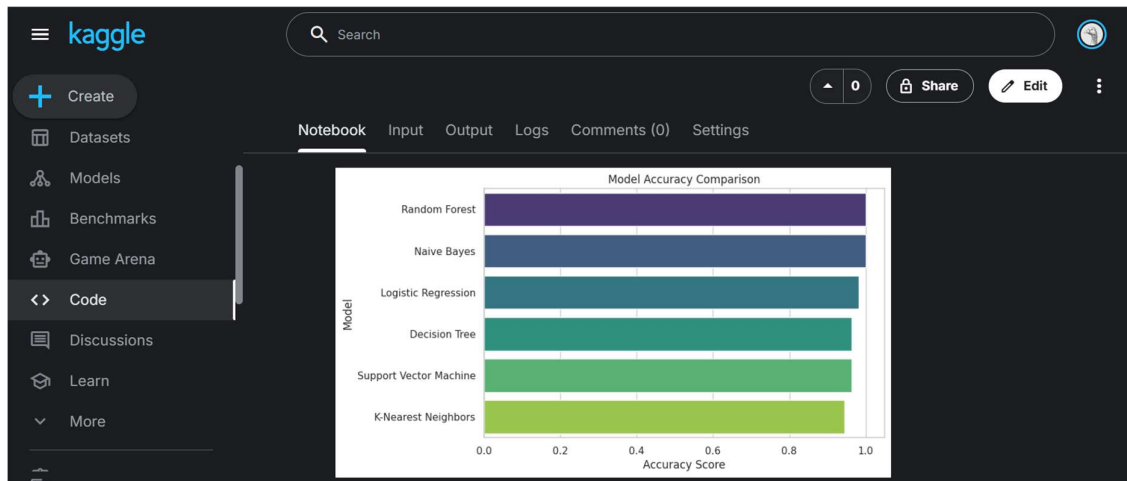


2.6 Comparison of Results

Model	Accuracy
Random Forest	~1.00
Logistic Regression	~0.98
Support Vector Machine	~0.98
Decision Tree	~0.96
K-Nearest Neighbors	~0.94
Naive Bayes	~0.90

From the comparison, **Random Forest** achieved the **highest accuracy (approximately 100%)**, closely followed by **Logistic Regression** and **SVM**.


```
In [14]: # Plot model accuracy comparison
sns.barplot(x='Accuracy', y='Model', data=results, palette='viridis')
plt.title("Model Accuracy Comparison")
plt.xlabel("Accuracy Score")
plt.ylabel("Model")
plt.show()
```



3. Link to code:

<https://www.kaggle.com/code/stephanicawuor/stephannie-lumumba-25097-classification-model>

4. Conclusion

This assignment demonstrated the process of building, evaluating, and comparing several classification algorithms on the same dataset.

Key insights include:

- **Random Forest** provided the best performance due to its ensemble nature and ability to reduce overfitting through averaging multiple decision trees.
- **Logistic Regression** and **SVM** also performed very well, indicating that the dataset is linearly separable to some extent.
- **Naive Bayes** had the lowest performance because its assumption of feature independence does not perfectly fit the correlated chemical features in the dataset.

Through this exercise, I learned how to:

- Perform exploratory data analysis (EDA) for classification problems
- Apply and compare multiple supervised learning models
- Use metrics like accuracy, precision, recall, and confusion matrices for evaluation
- Interpret model results and understand their strengths and weaknesses

Overall, this assignment enhanced my understanding of classification models and their practical application using Python and scikit-learn.